

INFO - 521
Introduction to Machine Learning
Final Project

German Bank Loan Defaulter Analysis

By

Abhay Kumara Sri Krishna Nandiraju



Table of Contents

Abhay Kumara Sri Krishna Nandiraju.....	1
Table of Contents.....	2
1. Introduction:.....	3
2. Methods and Materials:.....	5
a. Exploratory Data Analysis(EDA):.....	5
Exploring Categorical Variables:.....	6
Exploring Numerical Variables:.....	18
3. Results:.....	18
4. Discussion:.....	18
5. Conclusions:.....	18

1. Introduction:

The main objective of this project is to develop a machine learning model for the bank to predict whether the customer will default or not based on the historical data of various customers that did and did not default on their loan. It is crucial to predict loan defaulters as they can cause financial losses and affect the bank's ability to lend loans to other customers. Identifying loan defaulters helps banks perform risk management in an optimized manner. The German bank is facing issues with loan defaulters and wants to build a machine learning model to address the problem using a dataset that contains historical data of the customers who have taken loans from it. The dataset has the data of a thousand(1000) customers consisting of various attributes along with their respective default status indicating whether they defaulted on their loan or not.

The dataset has 17 columns and 1000 rows where the columns represent various attributes or features of the customers. Each row represents a customer along with their corresponding feature variables. Each customer is represented by the following 17 attributes:

- a. 'checking_balance': It represents the amount of money available in the customer's checking account.(DM- Deutsche Mark)
- b. 'months_loan_duration': It represents the duration since the loan was taken in months.
- c. 'credit_history': It represents the credit history of the customer.
- d. 'purpose': It represents the reason for which the loan was taken.
- e. 'amount': It represents the amount of loan taken.
- f. 'savings_balance': It represents the savings account balance..
- g. 'employment_duration': It represents the duration of employment.
- h. 'percent_of_income': It represents the loan installment amount as the percent of customer's income.

- i. 'years_at_residence': It represents the number of years the applicant has lived at their current residence.
- j. 'age': It represents the customer's age.
- k. 'other_credit': It represents the other existing credits.
- l. 'housing': It represents the type of housing owned by the customer.
- m. 'existing_loans_count': It represents the existing count of each customer's loans.
- n. 'job': It represents the customer's job type.
- o. 'dependents': It represents the number of dependents the customer has.
- p. 'phone': It represents whether the customer has a phone or not.
- q. 'default': It indicates whether the customer has defaulted on their loan or not.

The dataset has 17 columns out of which the 'default' column is the target or the response that needs to be predicted using the remaining variables or features. It is essential to check the number of categorical and numerical variables present in the dataset and in addition to this, each categorical and numerical variable's distribution should be computed to get an overview of each feature's distribution. It is also crucial to check the relationship between variables as it may help us in reducing the number of predictors required to train the machine learning models. Therefore, it is interesting to check the distribution of different variables along with the relationship between them.

2. Methods and Materials:

a. Exploratory Data Analysis(EDA):

The dataset has 1000 entries with 17 columns with no null values in any of the columns. Out of the 17 columns, 10 are categorical and the remaining 7 are numerical. Each of 10 columns consist of information regarding a categorical variable and each of the 7 columns consist of information about a numerical variable. The ten categorical variables are phone, housing, other_credit, job, savings_balance, purpose, checking_balance, credit_history, employment_duration and default. The seven numerical variables are months_loan_duration, amount, percent_of_income, years_at_residence, age, existing_loans_count and dependents. The categorical variable default is the response that needs to be predicted and the remaining variables are treated as predictors. The dataset has no missing values, null values and duplicate rows in it.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   checking_balance                      1000 non-null   object
1   months_loan_duration                 1000 non-null   int64
2   credit_history                       1000 non-null   object
3   purpose                             1000 non-null   object
4   amount                              1000 non-null   int64
5   savings_balance                     1000 non-null   object
6   employment_duration                 1000 non-null   object
7   percent_of_income                   1000 non-null   int64
8   years_at_residence                  1000 non-null   int64
9   age                                 1000 non-null   int64
10  other_credit                         1000 non-null   object
11  housing                             1000 non-null   object
12  existing_loans_count                 1000 non-null   int64
13  job                                 1000 non-null   object
14  dependents                          1000 non-null   int64
15  phone                              1000 non-null   object
16  default                             1000 non-null   object
dtypes: int64(7), object(10)
memory usage: 132.9+ KB

```

Fig-1: Dataset Information

Exploring Categorical Variables:

In this section, each categorical variable is explored by plotting pie-charts and count-plots. There are 700 non-defaulters and 300 defaulters in the dataset indicating the presence of class imbalance in the dataset. The dataset is biased towards customers that did not default. Out of all the customers, 596 possess a phone whereas 404 do not. Based on the job type there are 630 skilled workers, 200 unskilled workers, 148 management workers and 22 unemployed customers. 713 customers own a house whereas 179 customers live in a rented home. Surprisingly, 108 customers neither own a house nor live in a rented home. Around 814 customers did not take credit from anywhere but 139 customers took other credit from a bank and 47 customers took credit from a store.

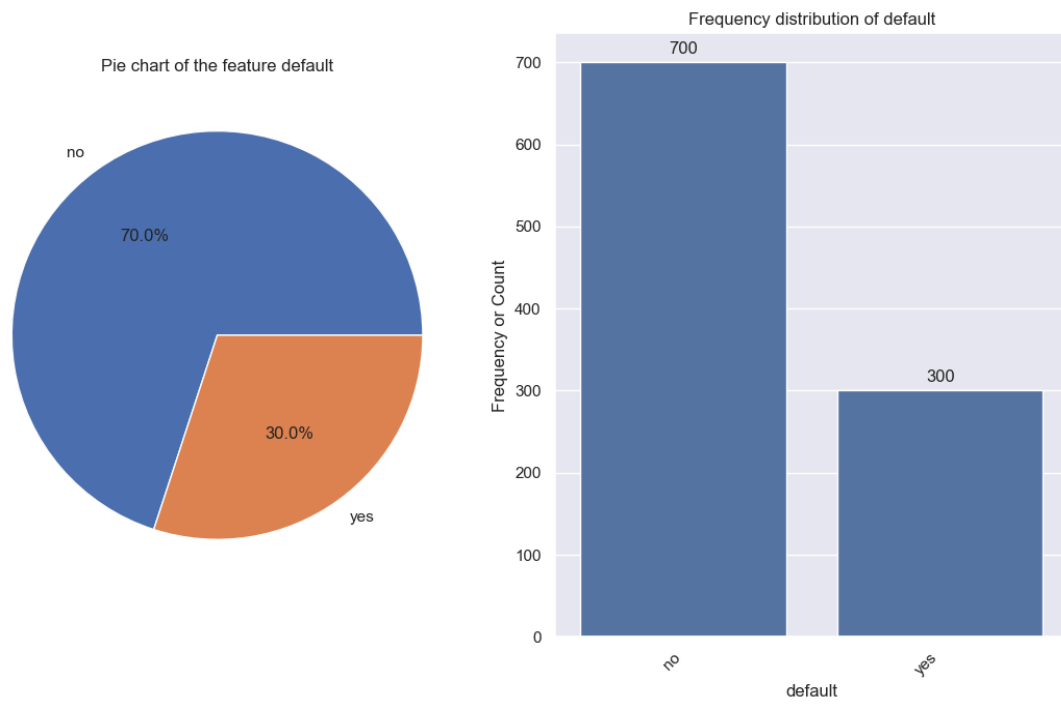


Fig-2: Countplot of the target or response default

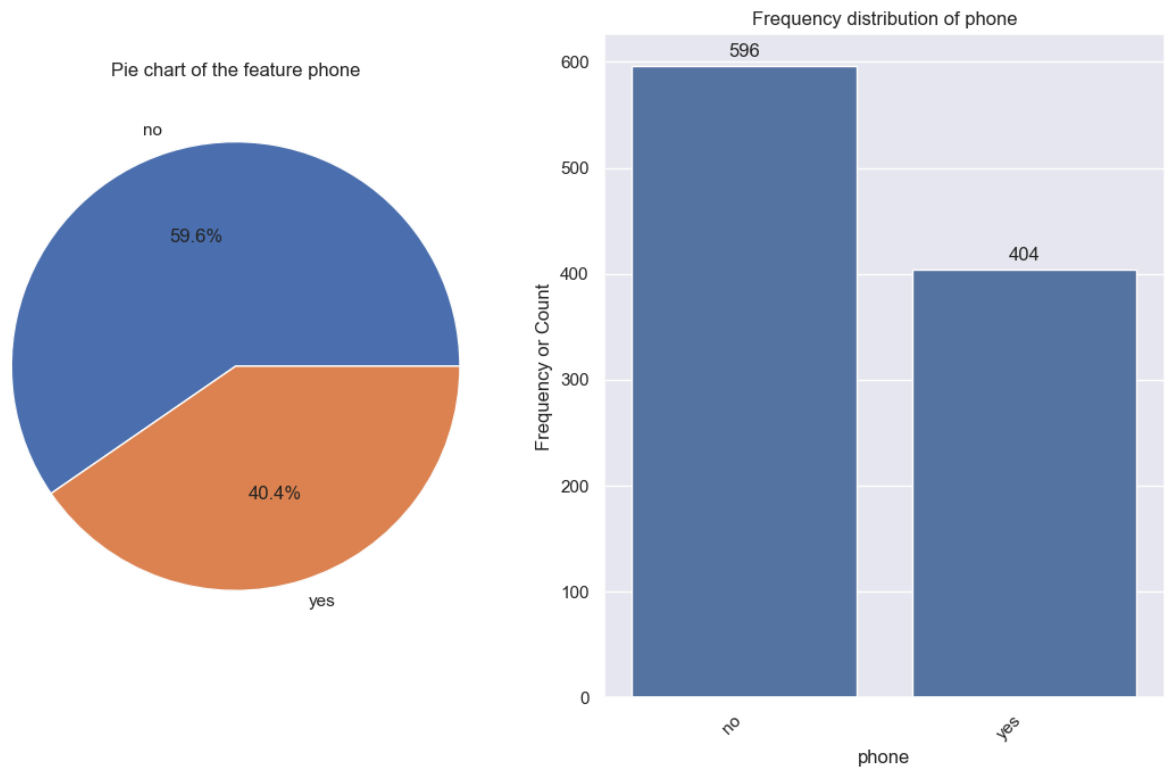


Fig-3: Countplot of predictor phone

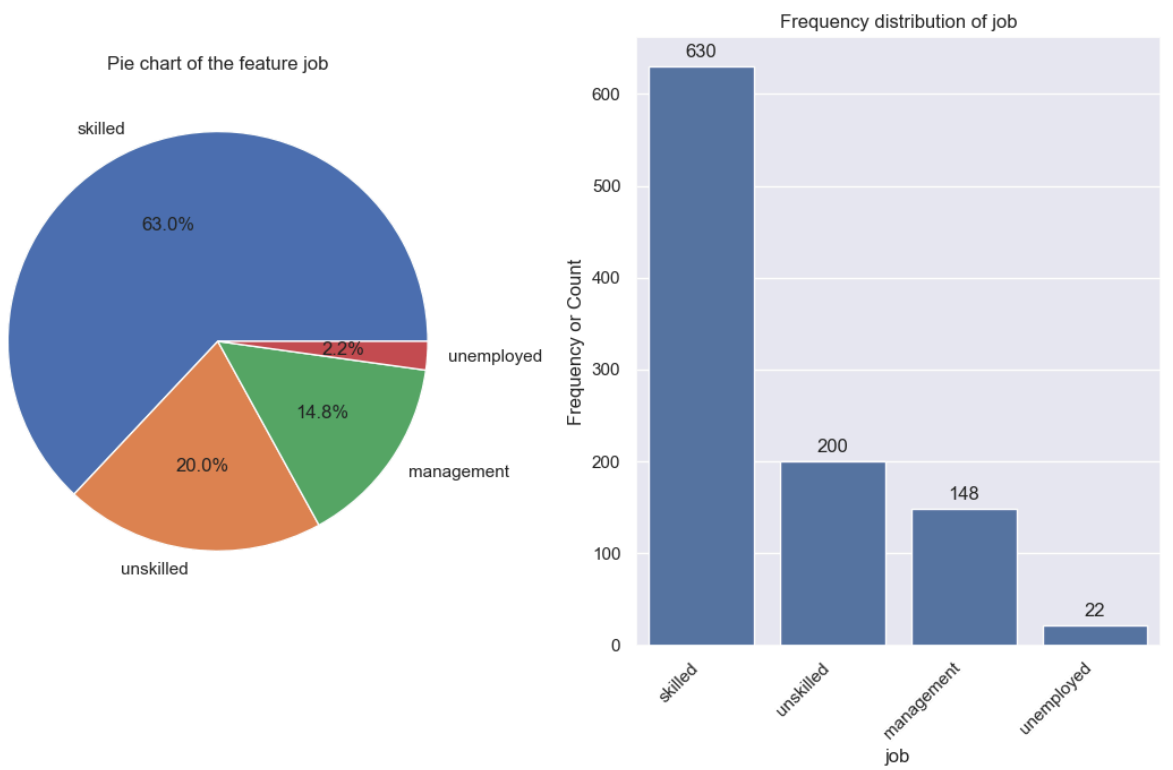


Fig-4: Countplot of predictor job

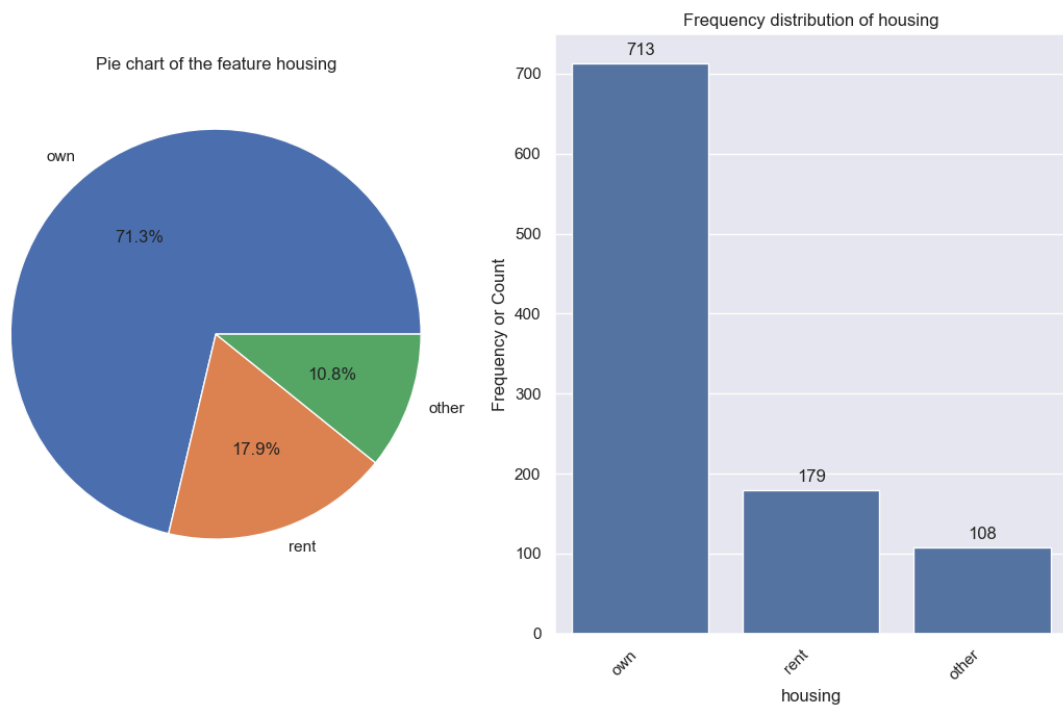


Fig-5: Countplot of predictor housing

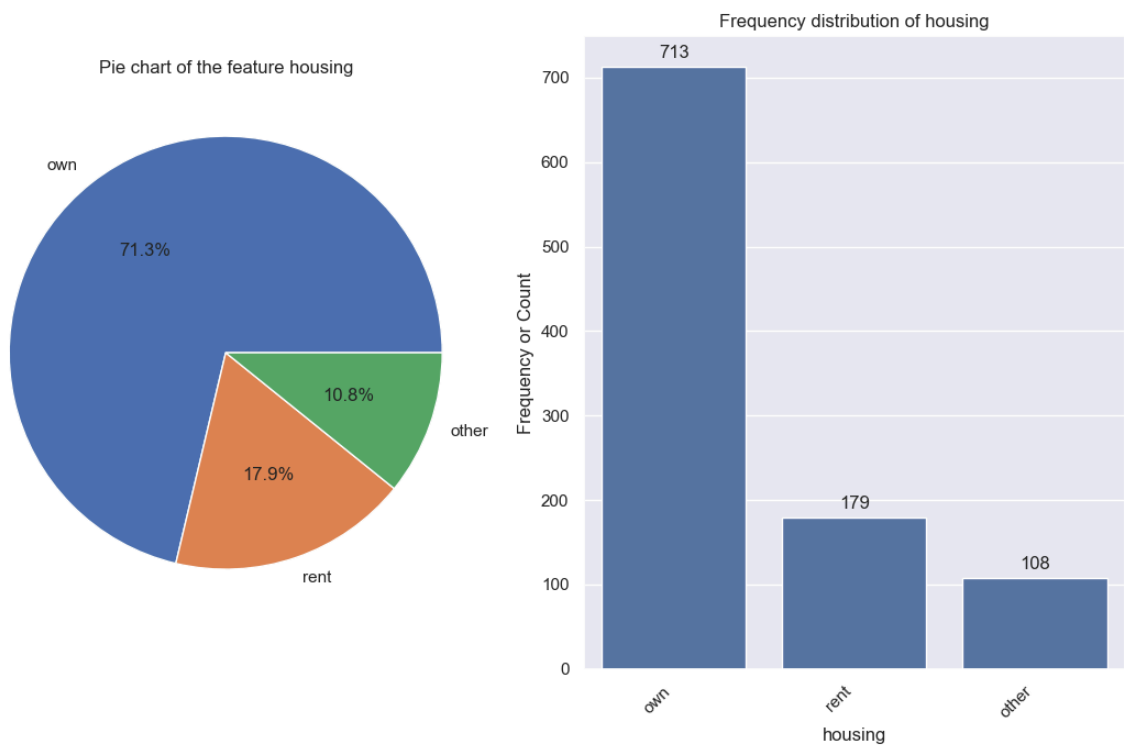


Fig-6: Countplot of predictor other_credit

There are 172 customers whose employment duration is less than 1 year, 339 customers whose employment duration is between 1 to 4 years, 174 customers whose employment duration is between 4 to 7 years, 253 customers whose employment duration is greater than 7 years and 62 customers who are unemployed. Large number of customers(603) have a savings balance less than 100 DM and few customers(63) have a savings balance between 500-1000 DM and only 48 customers have greater than 1000 DM. Moreover there are 183 customers with unknown savings balance and 103 customers with saving balance between 100 to 500 DM. The purpose column represents the purpose for which the loan was taken. There are two categories representing cars- one is named as 'car' whereas the other column is named as 'car0'. Both represent that the loan was taken to purchase a car but there seems to be a spelling mistake. Majority of customers(473) took loan to buy furniture/ appliances, whereas very few(22) took out loans for renovations. Around 349 customers(car + car0) took loan to buy a car and 97 customers took loan for business purpose but only 59 took loan for education purpose.

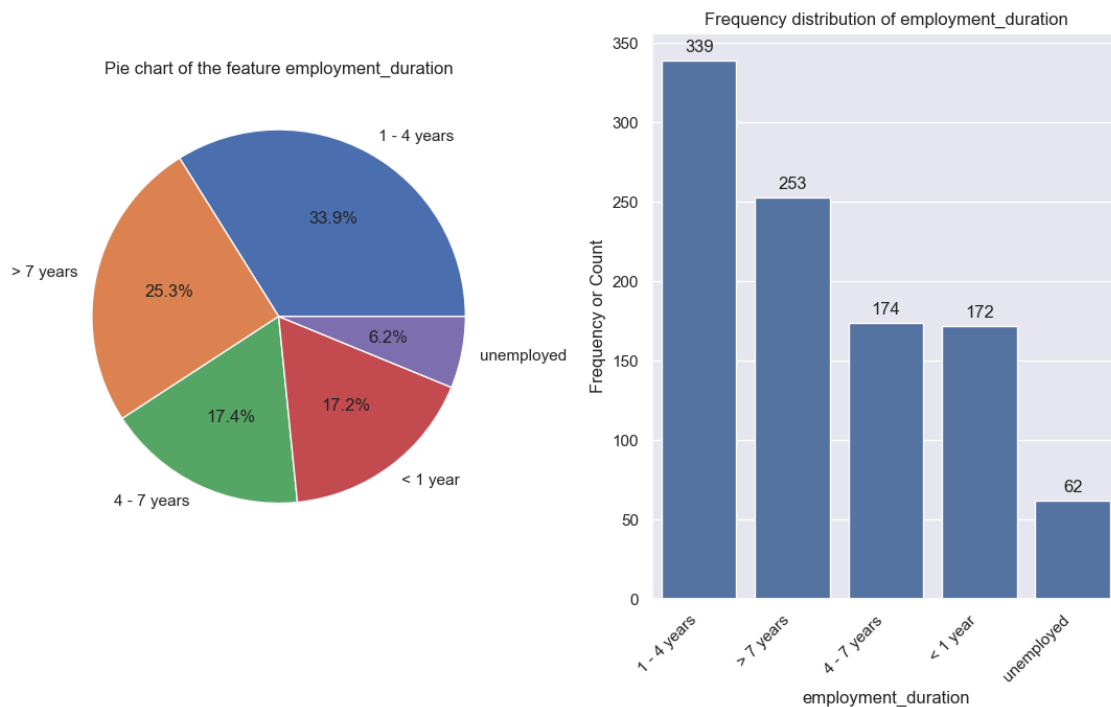


Fig-7: Countplot of predictor employment duration

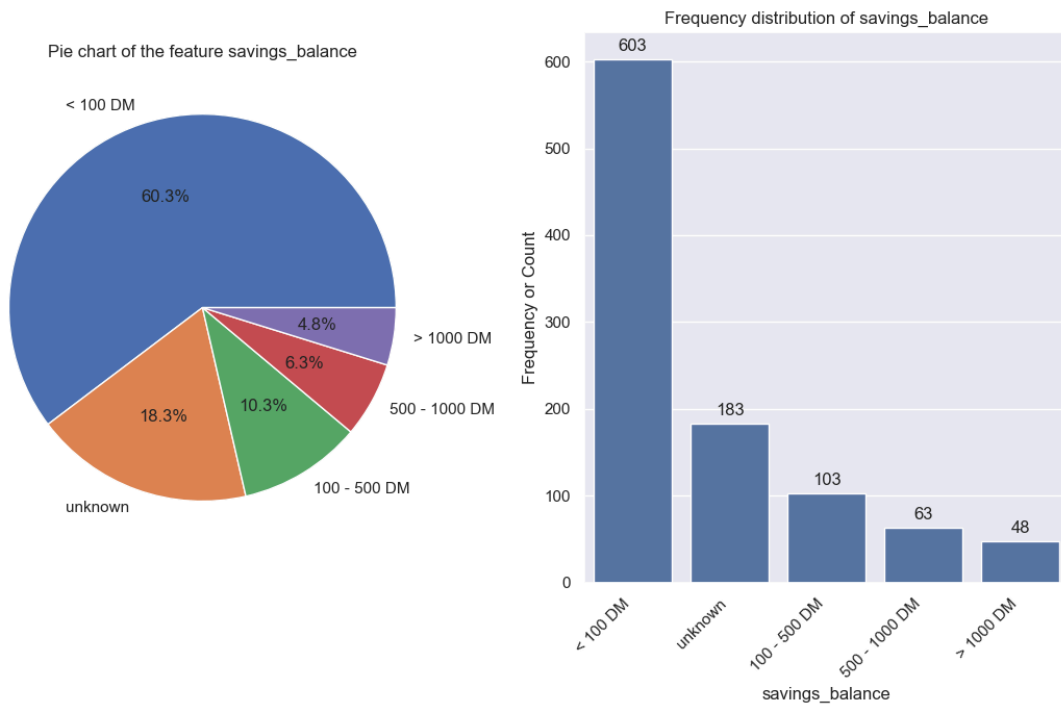


Fig-8: Countplot of predictor savings_balance

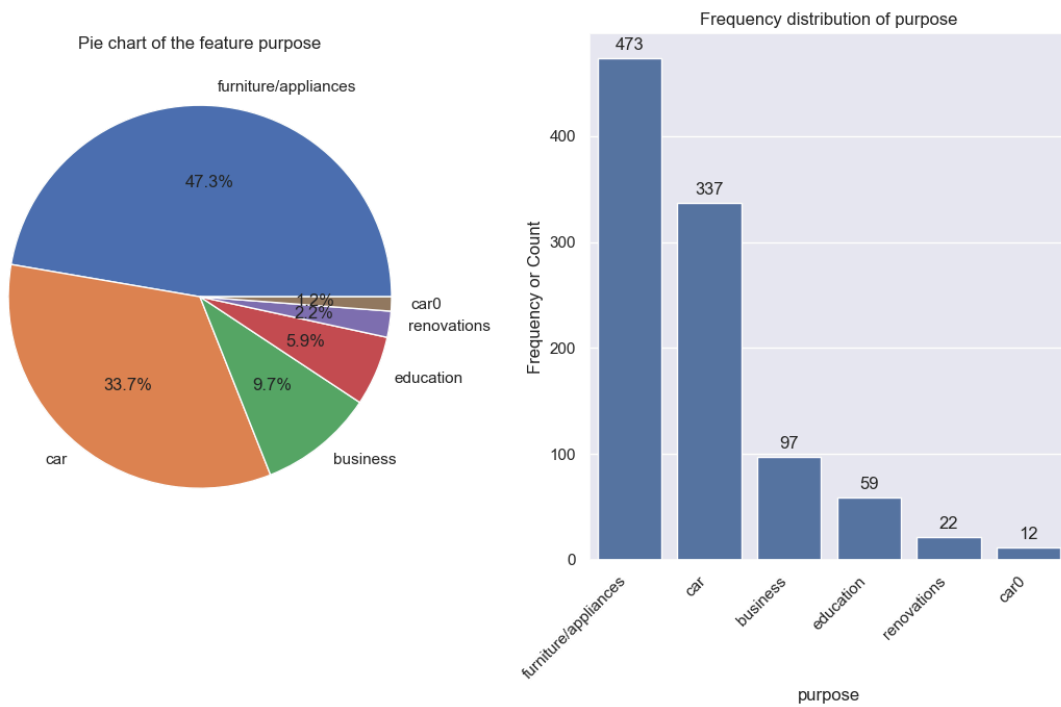


Fig-9: Countplot of predictor purpose

Majority of the customers(530) have a good credit history and only 40 have a perfect credit history. There are a significant number of customers(293) with critical credit history. Additionally, there are 88 customers with a poor credit history but only 49 with a very good credit history. There are 394 customers with unknown checkings balance, 274 customers with checking balance less than 0DM, 269 customers with checking balance between 1-200 DM and only 63 customers with checking balance greater than 200 DM.

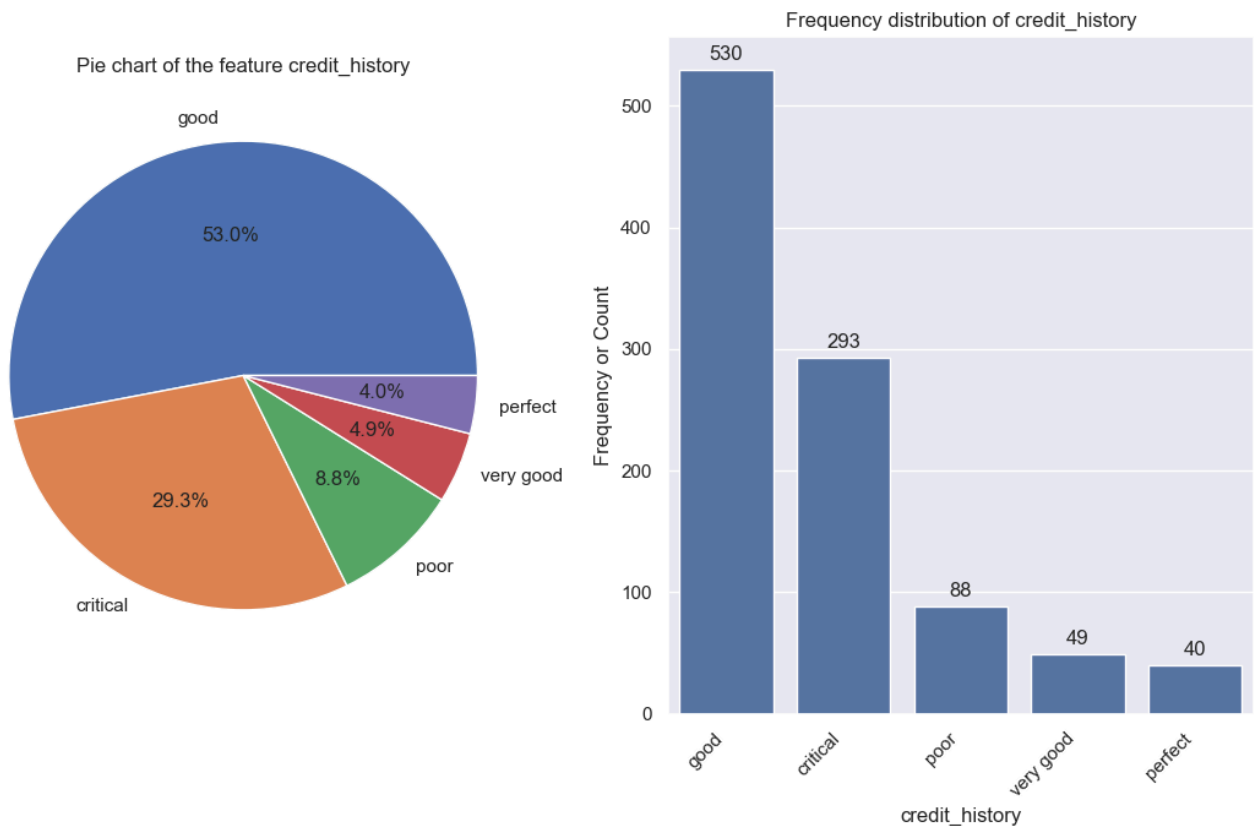


Fig-10: Countplot of predictor credit_history

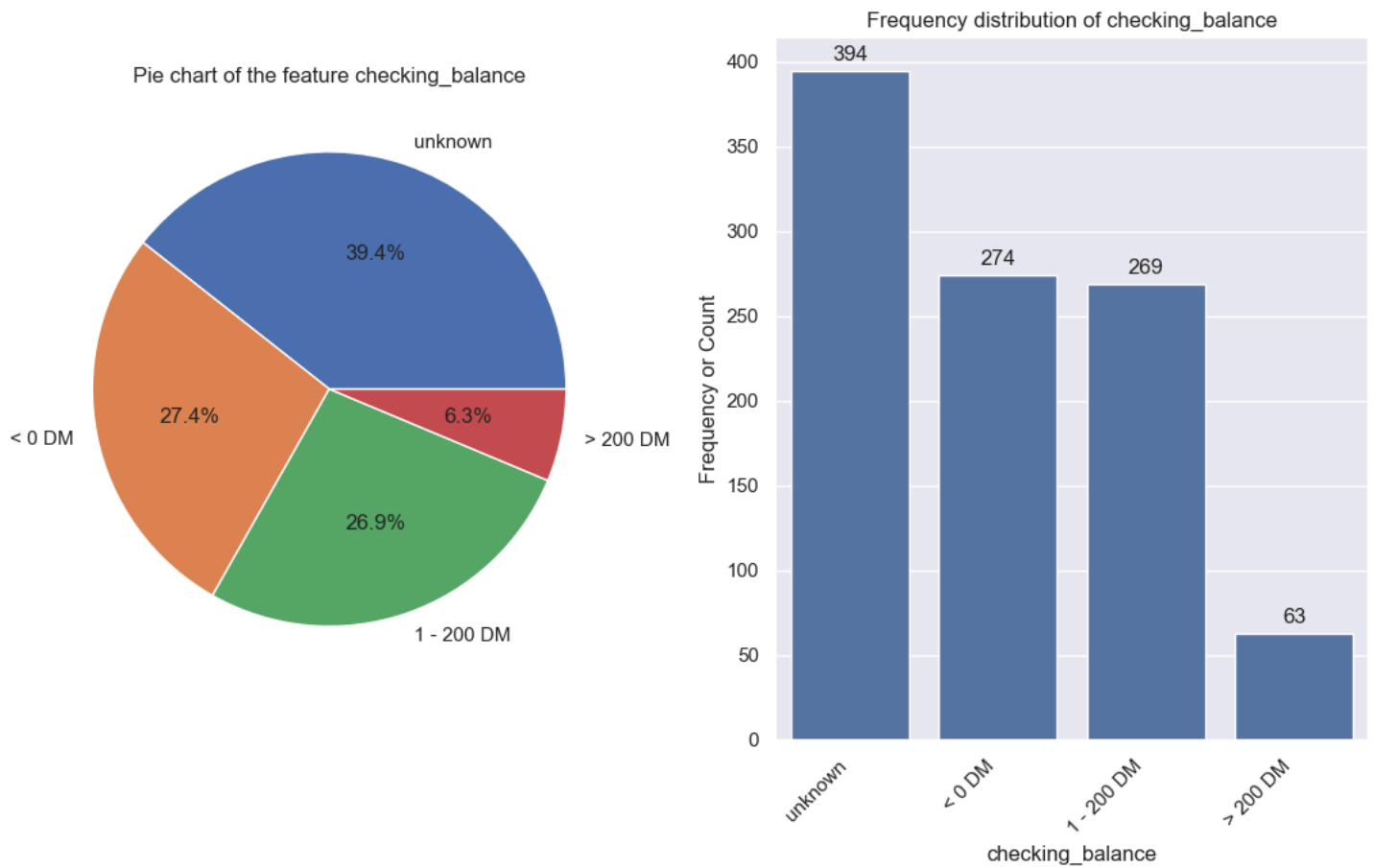


Fig-11: Countplot of predictor checking_balance

The above graphs illustrate the frequency distribution of each categorical variable whereas the graphs displayed below depict the category-wise percentage and category-wise number of defaulters and non-defaulters in each categorical variable or predictor.

In the below displayed category-wise plots, it is clear from the checking_balance graph that the percentage of defaulters increases as the checking_balance amount reduces. It is clear that customers belonging to the category '<0 DM' in the variable checking_balance have the highest number of defaulters whereas the customers belonging to the category 'unknown' have the least number of defaulters.

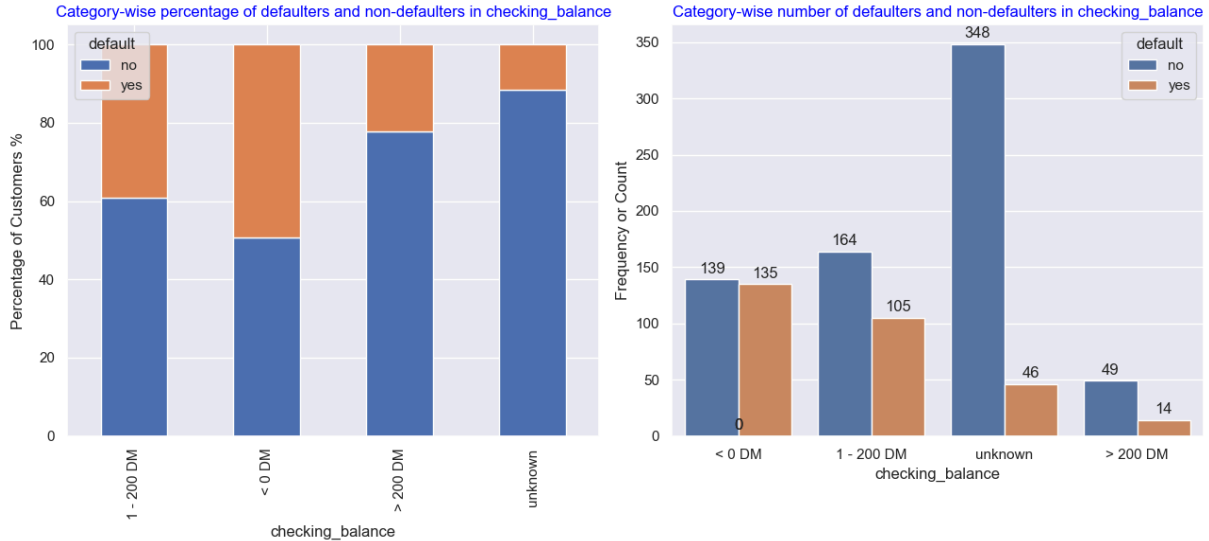


Fig-12: Category-wise plot of checking_balance

From the credit_history graph given below, it is surprising to see that the customers belonging to the categories 'perfect' and 'very good' default on their credit. This might have happened due to lack of adequate data about customers with 'perfect' and 'very good' credit history. Out of all the categories, the category 'good' has the highest number of non-defaulters(361) and defaulters(169). The category 'critical' has the lowest percentage of defaulters. Unemployed customers and customers whose employment duration is less than 1 year (<1 year) are more likely to default on their credit. Customers with more employment duration are less likely to default on their loan. Customers who took loan for the purpose of education and car(car + car0) are more likely to default.

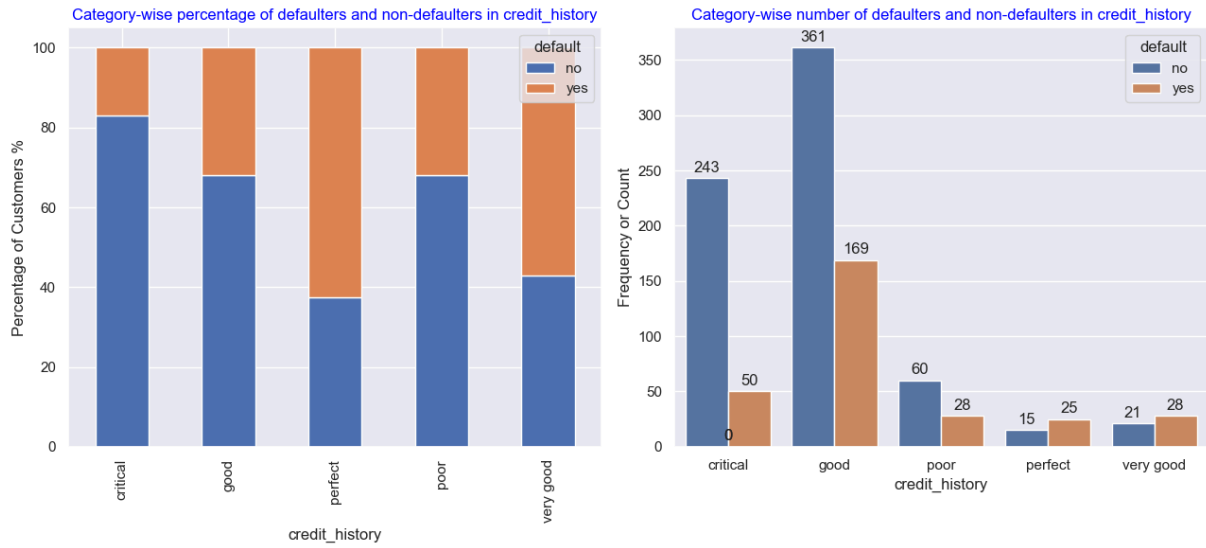


Fig-13: Category-wise plot of credit_history

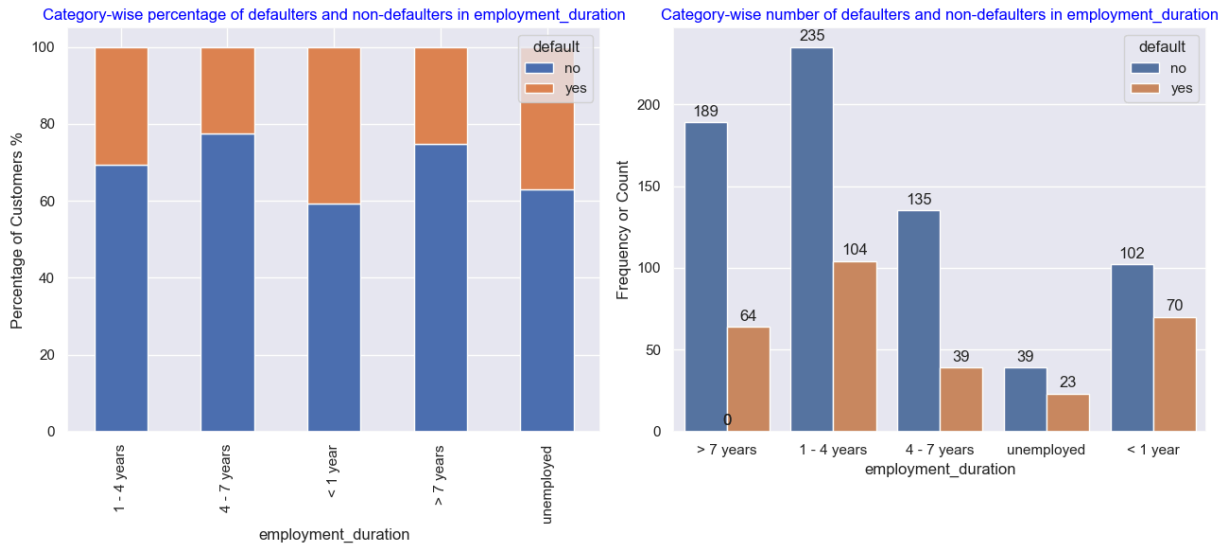


Fig-14: Category-wise plot of employment_duration

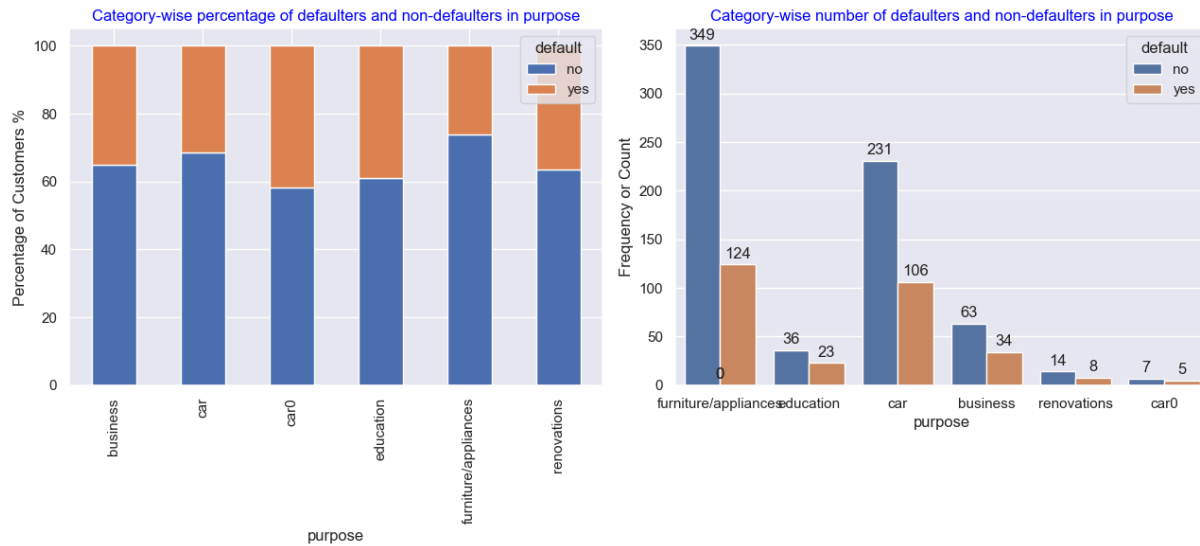


Fig-15: Category-wise plot of purpose

In the savings_balance figure, it is clear that the percentage of defaulters increases as the savings balance amount decreases. In addition to this, customers that own a house are less likely to default on their loan. Moreover, customers with no sources of other credit are less likely to default on their loan too. The percentage of defaulters in each of the categories of the predictors are approximately the same for the predictors phone and job.

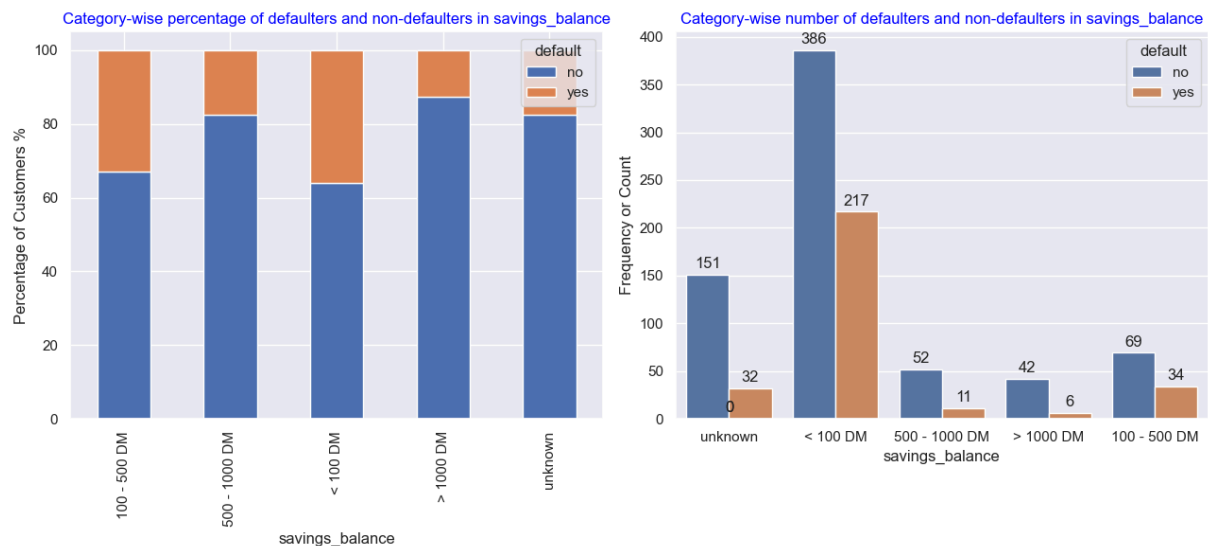


Fig-16: Category-wise plot of savings_balance

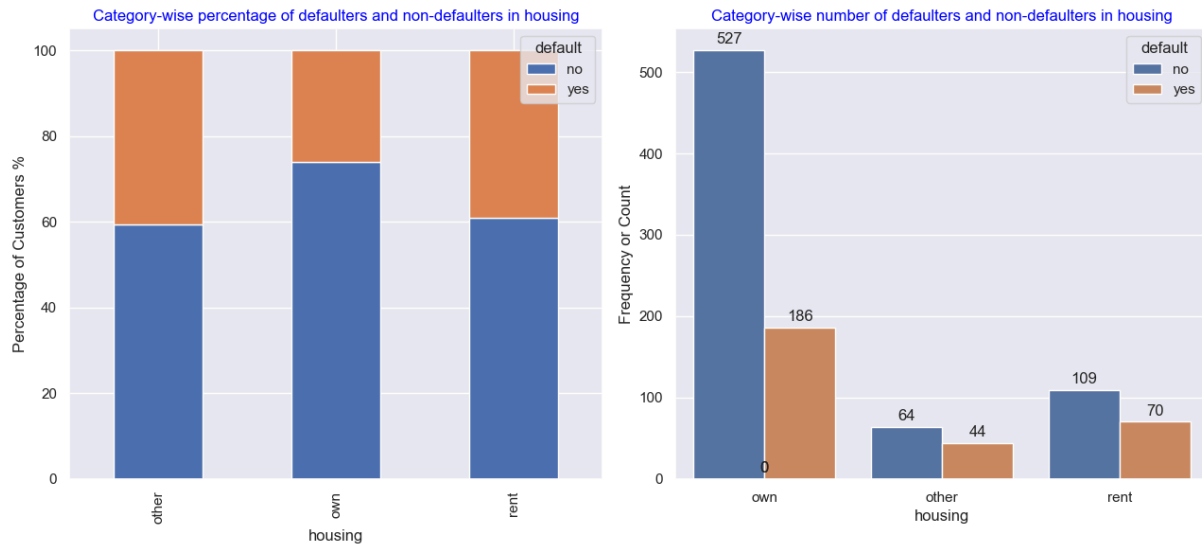


Fig-17: Category-wise plot of housing

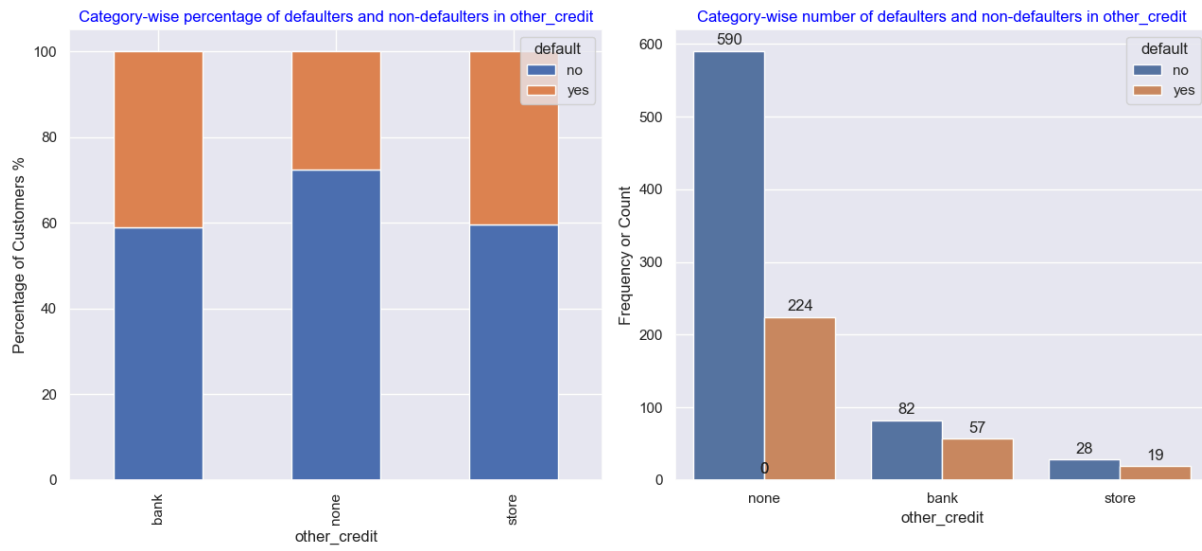


Fig-18: Category-wise plot of other_credit

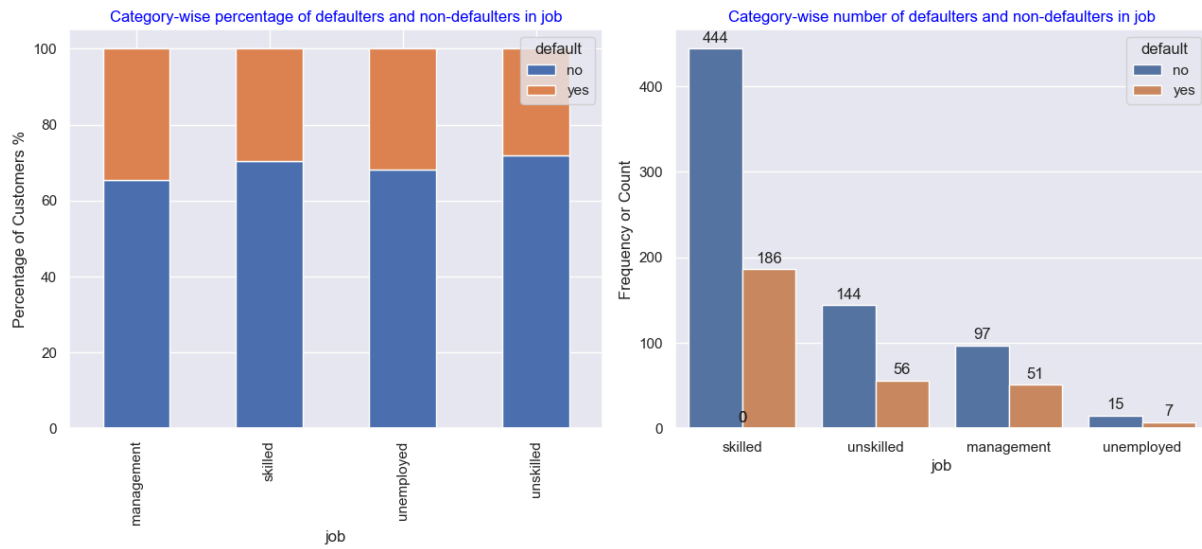


Fig-19: Category-wise plot of job

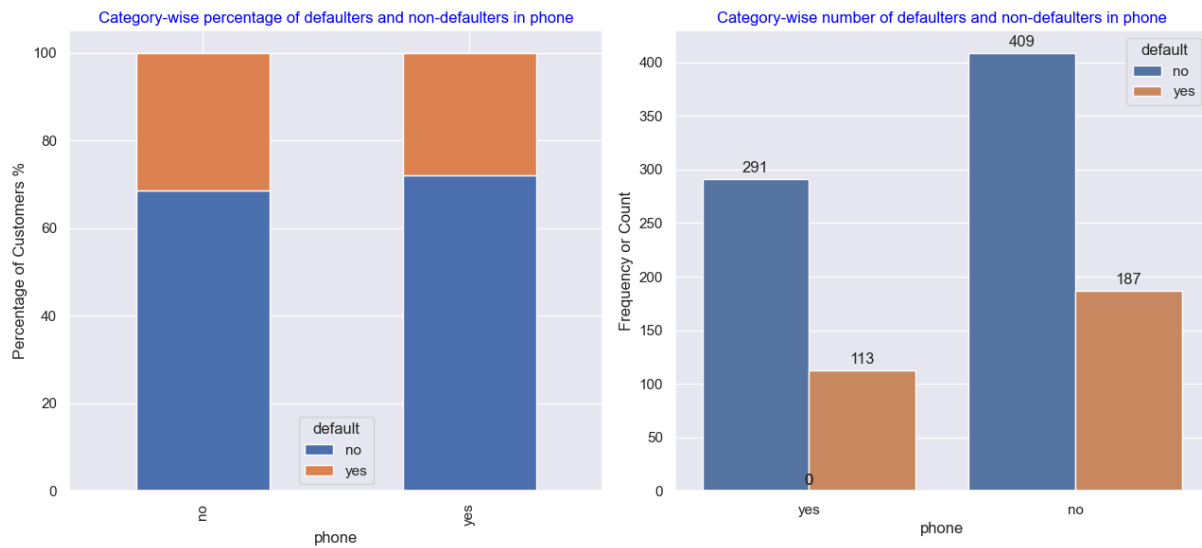


Fig-20: Category-wise plot of phone

Exploring Numerical Variables:

Numerical variables can be understood by plotting pair-plots and correlation matrix. Correlation matrix gives us a metric using which we can estimate the relationship between the variables. With the help of correlation value between two variables, we can determine if they are positively or negatively related to each other. The pairplots give us a visual depiction about how each variable is related to the other variables. From the correlation matrix and pairplot, it can be seen that the variables amount and months_loan_duration are moderately correlated with each other and moreover they are positively correlated. Remaining variables do not exhibit any correlation among them. The correlation coefficient between amount and months_loan_duration is 0.62 indicating they are positively correlated to each other.

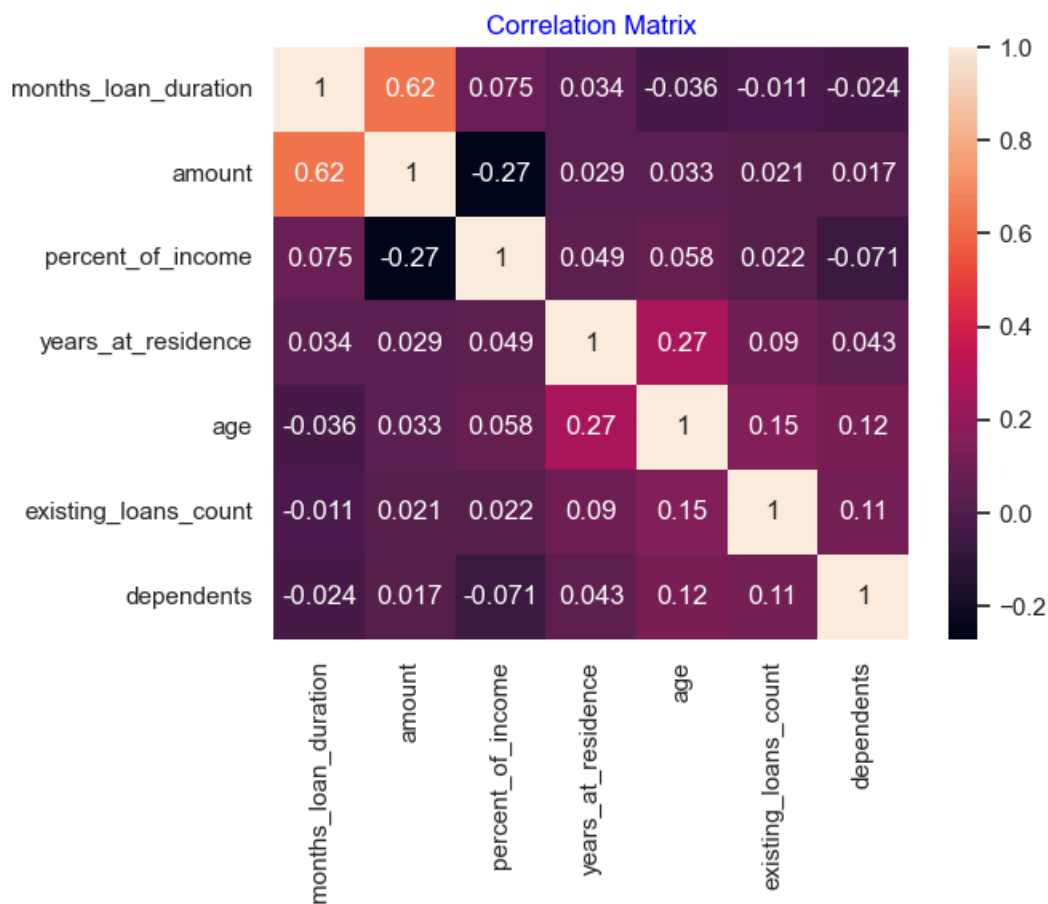


Fig-21:Correlation Matrix

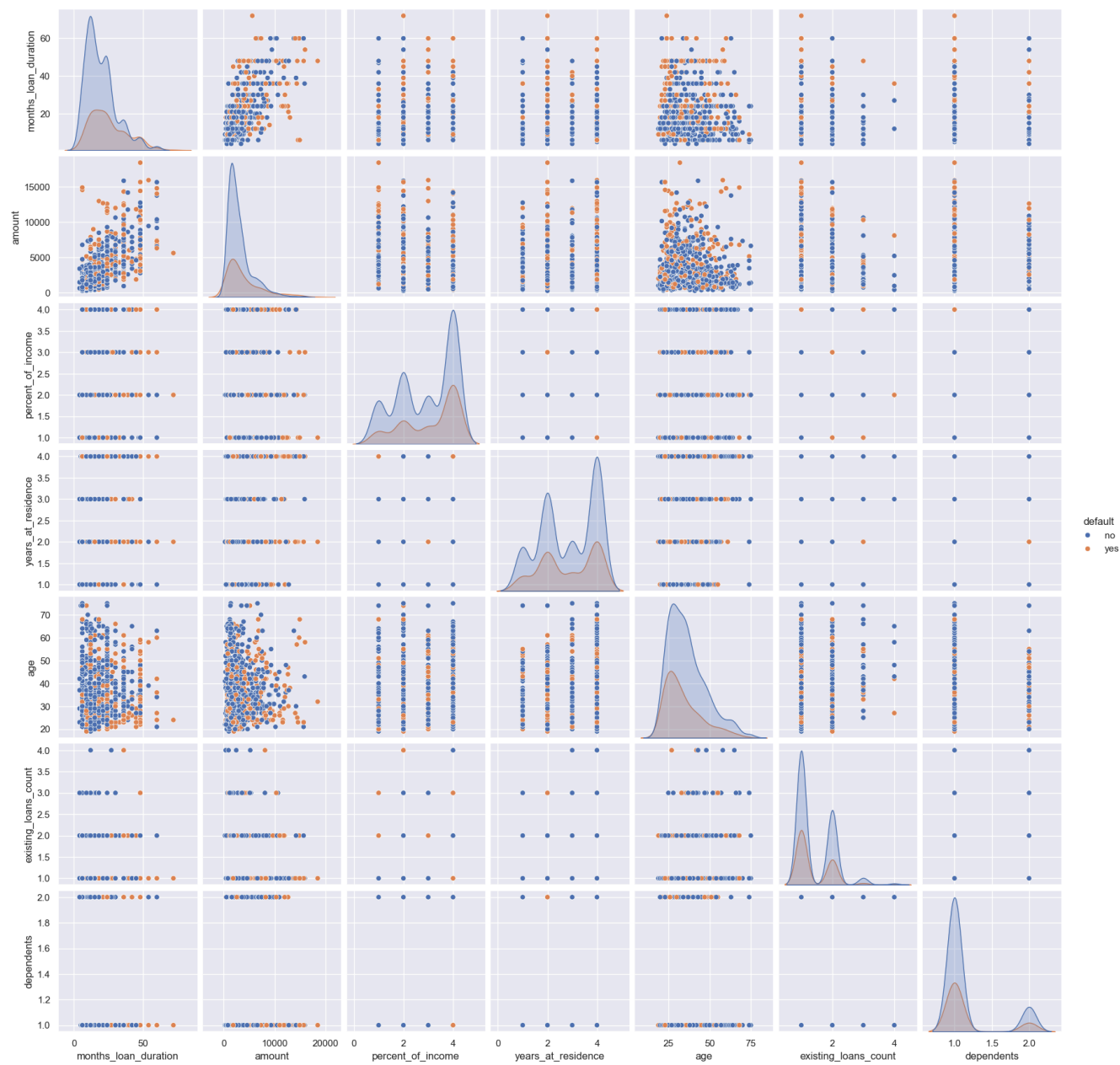


Fig-22: Pairplot between the variables

b. Pre-processing data for model training:

In the pre-processing step, firstly the categorical variable purpose has 'car' and 'car0' representing that the loan was taken to purchase a car. They both represent the same purpose but are misspelled. Hence, the category 'car0' in the predictor is renamed to 'car'. The numerical variable amount has a large range from 250 to 18424, hence it is normalized using the MinMaxScaler() function. The remaining numerical variables did not undergo any transformations. The categorical variables contain both ordinal and nominal variables. The nominal variables in the dataset are checking_balance, purpose, savings_balance, job, other_credit, housing, phone and the target default. Actually, checking_balance and savings_balance should be ordinal variables but they both have category 'unknown' in large numbers. Hence, it cannot be deleted and cannot be given any order. So, these two variables are considered nominal variables in the pre-processing step. The ordinal variables in the dataset are credit_history and employment_duration. The categories in variables credit_history and employment_duration are ordered as 'critical' < 'poor' < 'good' < 'very good' < 'perfect' and 'unemployed' < '<1 year' < '1-4 years' < '4-7 years' < '> 7 years' respectively.

The nominal variables in the dataset are encoded using a one-hot encoder and the ordinal variables are encoded using an ordinal encoder. The numerical variable amount is normalized using the min-max scaling method. The entire pre-processing steps are performed using the column-transformer pipeline to simplify the transformation process. The target column default has only two classes: yes and no. The class yes is mapped to 1 indicating loan defaulters whereas no is mapped to 0 indicating non-defaulters. The above said pre-processing steps are performed after splitting the data into train and test data in a 70:30 ratio. This is done to avoid data leakage problems and this method resembles the actual and practical scenario where the new data is not seen by the model. While training the XGBoost model on the transformed data, it was throwing an error because of the presence of '<' in some of the feature names. Hence, the required feature names were renamed in the transformed dataset obtained from pre-processing the raw, original dataset.

c. Model Tuning and Training :

The best model for this task is found out by carrying out cross-validation and hyperparameter tuning on supervised classifiers such as Bagging, Random Forest, Gradient Boosting Machine(GBM), AdaBoost and XGBoost. Additionally, Catboost and LightGBM models were also trained on the dataset to build advanced classifiers. The cross-validation scores were calculated on the basis of 5-fold cross-validation and recall was used as the scoring metric. The models are fine-tuned using the grid search cross validation approach. In the grid-search cross-validation approach, the user specifies a list of parameter values specific to the model and the algorithm performs a cross validation task on each and every combination of parameter values to find the best parameter setting for the given model.

After finding the best parameter values for a given model, the model is once again trained on the training set using the best parameters and is evaluated on a separate test set to check its performance. The metrics and results are visualized using the custom VisualizeMetrics class defined in the notebook. The VisualizeMetrics class has three methods in it. The calculate_metrics() method takes the model name, predictors and response as an input and computes the basic classification metrics such as recall, F1, accuracy, precision and AUC. The display_cm_roc() method takes the model name, predictors and response as arguments and returns the confusion matrix and ROC curve. The last method get_metrics() method is used to display the classification metrics of training set and test set, confusion matrix of the test set and the ROC curve of the test set. The class VisualizeMetrics was written to minimize the repetition of code.

3. Results:

4. Discussion:

5. Conclusions: